

2024 学年第一学期期末学业水平测试

高二年级技术试题卷（乙卷）

考生须知：

1. 答题前，请务必将自己的姓名、准考证号用黑色字迹的签字笔或钢笔分别填写在试题卷和答题纸规定的位置上。
2. 答题时，请按照答题纸上“注意事项”的要求，在答题纸相应的位置上规范作答，在本试题卷上的作答一律无效。
3. 非选择题的答案必须使用黑色字迹的签字笔或钢笔写在答题纸上相应区域内，作图时可先使用 2B 铅笔，确定后必须用黑色字迹的签字笔或钢笔描黑。

第一部分 信息技术（共 50 分）

一、选择题（本大题共 12 小题，每小题 2 分，共 24 分。每小题列出的四个备选项中只有一个是符合题目要求的，不选、多选、错选均不得分）

阅读下列材料，回答第 1 至 2 题：

百度地图 APP 可以获取和分析城市交通数据，辅助城市管理者优化交通流量，出行者可以优化出行路线，给人们生活带来了便利。

1. 根据材料，下列关于数据与信息的说法，正确的是

- A. 如第 1 题图所示，该截图是信息
- B. 系统处理的数据都是非结构化数据
- C. 出行过程中实际行驶时间与系统预计时间有差别，说明信息具有真伪性
- D. 系统根据实时信息和历史出行信息预测到达时间，说明信息具有可加工处理性

2. 根据材料，下列关于大数据的说法，正确的是

- A. 百度地图中获取的数据均属于图数据
- B. 百度地图中获取到的数据价值密度高，所以数据一定要准确无误
- C. 百度地图能对行程时间及道路状况进行预测，提供多条出行路线
- D. 百度地图中获取的数据都采用批处理计算进行处理

3. 下列关于信息编码的说法，正确的是

- A. 将声音数字化需要经过采样、量化、编码三个主要过程
- B. 二进制整数最左边增加 1 位后，奇偶性会发生变化
- C. 十六进制数 36 转换为二进制数，该二进制数中 1 的个数为 3
- D. 扫描某图片生成只有黑、白两种颜色的图像，每个像素至少用两位二进制数编码

阅读下列材料，回答第 4 至 5 题：

近年来，电动自行车违规进梯入户充电引发的安全事故屡有发生。为消除电梯安全隐患，开发了“梯智眼”系统，即在电梯内加装智能 AI 识别系统，对电动车进行识别。经过大量数据的训练，该系统的识别率达 99%，一旦检测到电动车则语音警报，电梯门无法关闭且自动停止运行。电梯控制系统将相关信息上传至服务器，实时将情况反馈给工作人员。

4. 根据以上材料，下列说法正确的是

- A. 该系统实现对电动车的识别属于行为主义方法
- B. 通过智能 AI 识别系统，能阻止所有电动车进入电梯
- C. “梯智眼”系统训练数据量越大，识别准确率一般会更高
- D. 人工智能对社会的发展起到了巨大的作用，可以忽略它对社会的威胁



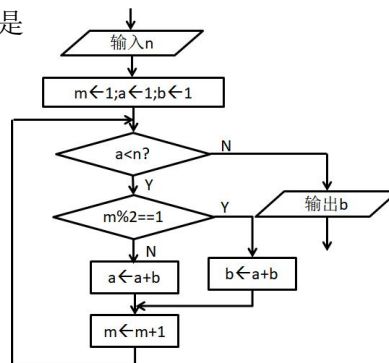
第 1 题图

5. 下列提升“梯智眼”系统数据安全所采用的措施，不合理的是

- A. 为所有的授权用户设置相同的密码
- B. 为提高服务器中数据的保密性，做好数据加密工作
- C. 为保证数据的完整性，需要进行数据校验
- D. 定期备份服务器中数据，做好数据备份

6. 某算法的部分流程图如右图所示。执行这部分流程，若输入 n 的值为 20，则输出结果是

- A. 8
- B. 13
- C. 21
- D. 34



第 6 题图

7. 六岁以上人群在安静状态下的正常心率通常为 60~100 次/分钟。低于 60 称为心动过缓，高于 100 称为心动过速。体检时将测得的某人心率存入变量 rate，下列 Python 程序段不能正确输出心率状态的是

- | | | | |
|-------------------|-------------------|-------------------|-----------------------|
| A. sta = "心动过缓" | B. if rate > 100: | C. sta = "心动过速" | D. if rate < 60: |
| if rate > 100: | sta = "心动过速" | if rate < 60: | sta = "心动过缓" |
| sta = "心动过速" | elif rate < 60: | sta = "心动过缓" | if rate > 100: |
| elif rate <= 100: | sta = "心动过缓" | elif rate <= 100: | sta = "心动过速" |
| sta = "正常" | else: | sta = "正常" | if 60 <= rate <= 100: |
| print(sta) | sta = "正常" | print(sta) | sta = "正常" |
| | print(sta) | | print(sta) |

8. 某队列从队首到队尾的元素依次为 30, 71, 15, 82, 13, 61。若队首元素大于等于 60 出队，小于 60 出队后重新入队。入队或出队各计一次操作，经过 N 次操作后，队列中剩余元素全为小于 60 的数。则 N 的值为

- A. 6
- B. 7
- C. 8
- D. 9

9. 某二叉树的数组表示示意图如第 9 题图所示，其根节点在后序遍历序列、中序遍历序列中的位置序号分别记为 x, y，则 x-y 的值为

0	1	2	3	4	5	6	7	8	9
A	B	C		D	E				F

第 9 题图

- A. 1
- B. 2
- C. 3
- D. 4

10. 有如下 Python 程序段：

```

s1, s2 = "newpython", "132"
n1, n2, res = len(s1), len(s2), ""
i = j = 0; m = 0
while i < n1:
    x = int(s2[j]) - 1
    res = s1[m*n2+x] + res
    i += 1; j = (j+1)%n2; m = i//n2
  
```

执行以上程序段后，变量 res 的值是

- A. "nweptyhno"
- B. "nwehnopty"
- C. "onhpytnwe"
- D. "onhytpewn"

11. 有如下 Python 程序段:

```
#读取 n 个数据, 存入列表 a, 代码略
tmp = 1; ans = 0
for i in range(1, len(a)):
    if a[i] >= a[i - 1]:
        tmp = tmp + 1
    else:
        if tmp > ans:
            ans = tmp
        tmp = 1
```

若列表 a 有 6 个元素, 运行该程序段后 ans 的值为 3, 则列表 a 的值可能的是

- A. [2, 4, 4, 6, 3, 1] B. [5, 9, 8, 2, 4, 6] C. [6, 5, 5, 9, 2, 1] D. [1, 3, 2, 2, 1, 6]

12. 有如下 Python 程序段:

```
from random import randint
s = [0]*10
a = [5, 4, 7, 8, 4, 3, 6, 2]
top = 0; s[top] = a[0]; b = ""
n = randint(5, len(a)-1)
for i in range(1, n):
    while top != -1 and a[i] > s[top]:
        top -= 1
    top += 1
    s[top] = a[i]
while top != -1:
    b = b + str(s[top])
    top -= 1
print(b)
```

程序运行后, 输出的结果可能是

- A. 786 B. 268 C. 68 D. 86

二、非选择题 (本大题共 3 小题, 其中第 13 小题 7 分, 第 14 小题 10 分, 第 15 小题 9 分, 共 26 分)

13. 某停车场是一个矩形区域, 共有 m 排, 每排有 n 个连续的停车位, 编号依次为 1~m*n, 可用于停放小型车和大型车, 如第 13 题图 a 所示。小型车 (A 型) 占 1 个停车位, 大型车 (B 型) 需占 2 个相邻的停车位。编写 Python 程序, 根据停车场的当前状态以及已完成的停车或离开操作, 输出当前空闲的总停车位数量和还可以停放的大型车最多数量。请回答下列问题:

(1) 若 m=2, n=10, 开始时停车场全空, 经过如第 13 题图 b 所示的操作后, 还可停放大型车的数量最多为 ▲ 辆。

(2) 实现上述功能的部分 Python 程序如下, 请在划线处填入合适的代码。

```
m = 5; n = 10
sta = [[False for i in range(n)] for j in range(m)]
,,,
```

读取当前停车场车位状态数据, 存入二维数组 sta, 代码略

若 sta[i][j] 的值为 True 表示该车位已停车, 值为 False 表示该车位未停车

统计当前总空位数, 存入变量 cnt1, 代码略

'''

while True:

'''

读取车位感应装置采集的数据：汽车类型(A或B)、停车或离开状态(True或False)、车位编号起始值，存入 car、stop 和 num，代码略

'''

if car == "A":

w = 1

else:

w = 2

if stop == True:

cnt1 -= w

else:

①

i = (num-1)//n

②

sta[i][j] = not sta[i][j]

if car == "B":

sta[i][j+1] = not sta[i][j+1]

cnt2 = 0

for i in range(m):

j = 0

while ③:

if not sta[i][j] and not sta[i][j+1]:

j += 1

cnt2 += 1

j += 1

print("当前总空余车位数：", cnt1, "，还可停放大型车的最大数量：", cnt2)

14. 空气质量指数(AQI)分为六个等级，分别对应不同的空气质量，如第14题图a所示。

老王属于环境敏感人群，当AQI等级 ≥ 3 时，老王就得减少户外的活动时间。为了解决这个问题，每当老王居住地空气污染较严重时，他便会选择其他空气质量指数较好的地区作为自己的旅游地。为了方便选择旅游地，他会对该地区的空气质量情况进行简单分析，再进行相应的选择。

AQI 值	AQI 等级	空气质量
0-50	1	优
51-100	2	良
101-150	3	轻度污染
151-200	4	中度污染
201-300	5	重度污染
300 以上	6	严重污染

第14题图a

日期	最高温度	最低温度	AQI值	AQI等级	空气质量
2024-01-01	3	-2	59	2	良
2024-01-02	2	-3	49	1	优
2024-01-03	2	-5	28	1	优
2024-01-04	0	-5	50	1	优
2024-01-05	3	-4	625	1	严重污染
2024-01-06	2	-5	49	1	优
2024-01-07	3	-6	29	1	优
2024-01-08	2	-8	48	1	优
2024-12-26	2	-1	283	5	重度污染
2024-12-27	0	-2	217	5	重度污染
2024-12-28	4	-3	59	2	良
2024-12-29	4	-2	172	4	中度污染
2024-12-30	2	-2	20	3	优
2024-12-31	4	1	93	2	良

第14题图b

请回答下列问题：

(1) 第 14 题图 b 的数据是某地区一年的天气数据，存储在 “AQI.xlsx” 文件中，可以看出，2024-01-05 当天的 AQI 值与 1 月份其他各日有较大偏差，这属于常见数据问题中的 ▲（单选，填字母：A. 数据异常/B. 逻辑错误/ C. 格式不一致）。

编写 Python 程序实现上述功能，主程序如下：

```
#导入相关模块及设置对齐方式等，代码略
df = pd.read_excel("AQI.xlsx")
print(countDays(df))
date = input("请输入日期(格式为 yyyy-mm-dd):")
date_AQI(df,date)
```

(2) 编写统计天数的自定义函数 countDays()。根据该地区天气数据，统计 2024 年该地区各月份空气质量为优或良的天数，输出数据如第 14 题图 c 所示。请在划线处填入合适的代码。

```
def countDays(df1):
    df1["月份"] = ""
    for i in df1.index:
        tmp = df1.at[i,"日期"]
        df1.at[i,"月份"] = ①
    df_1 = df1[②]
    df_s = df_1.groupby("③",as_index=False).AQI 等级. ④
    df_s.rename(columns={"AQI 等级":"天数"},inplace=True) #修改列标题
    g = df_s.sort_values("天数",ascending=False)
    return g
```

	月份	天数
5	06	16
0	01	15
3	04	12
4	05	12
2	03	9
9	10	9
1	02	7
7	08	7
8	09	5
10	11	4
11	12	4
6	07	3

第 14 题图 c

(3)编写自定义函数 date_AQI()，加框处代码能实现输出某一天空气质量情况的选项有 ▲（多选，填字母）。（注：全部选对的得 2 分，选对但不全的得 1 分，不选或有错的得 0 分）

```
def date_AQI(df,date):
```

A. for i in len(df): if df.at[i,"日期"]==date: print(df[i:i+1])	B. for i in df.index: if df.at[i,"日期"]==date: print(df[i])
C. print(df[df["日期"]==date])	D. for i in df.values: if i[0]==date: print(i)

15. 某工厂经常有多种不同类型的原料物资需要入库，为实现物资高效存储和管理，仓库管理员采用一种优化的物资存储方案，目标是尽量将同类物资存放在同一个货架上，以便后续清点及出库等操作。

物资入库操作方法如下：

- ①将当前数量最多的同种类物资放置到与货架剩余容量最接近且能全部放下的货架上；
- ②若不能全部放下，则选择剩余容量最大的货架放置，剩余部分等待后续放置；
- ③重复上述放置操作，直至全部放完或货架全满为止。

编写 Python 程序，模拟物资入库上架过程。程序运行时从文件中读取物资数据及各货架剩余容量数据，计算并输出各物资放置信息。请回答下列问题：

(1) 若物资数量、货架剩余容量数据如下表所示：

物资类型	A	B	C	D	E	货架编号	C1	C2	C3
物资数量(件)	150	300	280	200	250	剩余容量(件)	250	600	400

则货架 C3 中放置的物资类型为 ▲ (填物资类型)

(2) 编写自定义函数 merge(Lst)，对类型相同的物资进行合并。参数 Lst 为物资列表，列表每个元素分别由物资类型和数量两项组成。例如，`[["A", 400], ["B", 500], ["A", 300], ["C", 200]]` 合并后为 `[["A", 700], ["B", 500], ["C", 200]]`。Python 程序代码如下：

```
def merge(Lst):
    i = 0; n = len(Lst)-1
    while i < n:
        j = i + 1
        while j <= n:
            if Lst[i][0] == Lst[j][0]:
                Lst[i][1] += Lst[j][1]
                Lst[j] = Lst[n]
                j -= 1 ; n -= 1
            j += 1
        i += 1
    return Lst[:n+1]
```

若漏写加框处代码，会导致某些情况下无法得到函数功能的结果。调用 merge(Lst) 函数，下列 4 组数据中能测试出这一问题的是 ▲ (单选，填字母)。

- A. `[["A", 100], ["B", 500], ["C", 300], ["A", 200], ["D", 200]]`
- B. `[["A", 400], ["B", 300], ["B", 200], ["C", 250], ["B", 100]]`
- C. `[["A", 300], ["B", 100], ["C", 300], ["D", 100], ["B", 300]]`
- D. `[["A", 400], ["B", 500], ["C", 100], ["E", 200], ["D", 100]]`

(3) 实现模拟物资入库上架的 Python 程序代码如下，请在划线处填入合适的代码。

```
def find(x, shelf):
    k = -1 ; d = float("-inf")          #表示负无穷
    for i in range(len(shelf)):
        if shelf[i][1] == 0:
            continue
        if x == shelf[i][1]:
            return i
```

```

        elif x < shelf[i][1]:
            if ①:
                d = shelf[i][1] - x; k = i
            else:
                if d < 0 and shelf[i][1] - x > d:
                    d = shelf[i][1] - x; k = i
        return k
    ,,,

```

读取物资数据和货架状态数据, 分别存入列表 data 和列表 shelf 中。列表 shelf 的每个元素包含 2 个数据项, 分别为货架编号和剩余容量, 代码略

```

    ,,,
    data = merge(data)
    #对列表变量 data 按数量降序排序, 代码略
    n = len(data)
    for i in range(n):
        data[i].append(i+1)          #为 data[i]追加一个元素
    data[i][2] = -1
    head = 0; ans = []               #记录物资存放信息
    while head != -1:
        k = find(data[head][1], shelf)
        if k == -1: break
        if data[head][1] <= shelf[k][1]:
            ②
            ans.append([shelf[k][0], data[head][0], data[head][1]])
            head = data[head][2]
        else:
            ans.append([shelf[k][0], data[head][0], shelf[k][1]])
            data[head][1] -= shelf[k][1]
            shelf[k][1] = 0
            p = head; q = data[p][2]
            while q != -1 and data[head][1] < data[q][1]:
                p = q ; q = data[q][2]
            if p != head:
                tmp = head
                head = data[head][2]
                data[p][2] = tmp
            ③
    if head != -1:
        #输出无法入库的剩余物资信息, 代码略
    #根据 ans 列表, 输出入库信息, 代码略

```